# Loops (`while`, `do-while`, `for`) and Functions

Computer Programming 1 · Lecture 5

Siyavushkhon Kholmatov · New Uzbekistan University

Week 5 · Fall 2026

# Where we are

**Last week**
The conditional operator `?:`, `switch` / `case`, and the first loop — `while`, with `break` and `continue`.

**Today**
The other two loops, nested loops — and functions (P1).

**Next week**
Functions: arguments, overloading, and passing by value or by reference.

# `while` — a quick recap

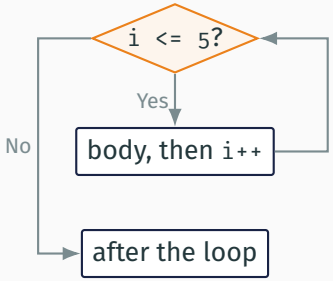
# The three parts, in every loop

```cpp
int i = 1;              // initialise

while (i <= 5) {        // test
    cout << i << " ";
    i++;                // update
}
cout << "| i = " << i;
```

**Output**

```
1 2 3 4 5 | i = 6
```




The test runs before every pass — so a `while` loop may run **zero** times.

# Counters and accumulators

```cpp
int n;
cin >> n;

int total = 0;       // sum of nothing
int i = 1;

while (i <= n) {
    total += i;
    i++;
}
cout << "Sum = " << total << endl;
```

| Pass | i | total |
|---|---|---|
| before | 1 | 0 |
| 1 | 2 | 1 |
| 2 | 3 | 3 |
| 3 | 4 | 6 |
| 4 | 5 | 10 |

With `n = 4` it prints `Sum = 10`; with `n = 0` the body never runs and it prints `Sum = 0`. ✓

# The three ways to loop forever

### No update

```
int i = 1;
while (i <= 5) {
    cout << i;
}
```

`i` never changes.

### Test never exactly met

```
int i = 0;
while (i != 10) {
    i += 3;   // 0 3 6 9
     12
}
```

Use `<` or `<=`, not `!=`.

### Stray semicolon

```
int i = 1;
while (i <= 5);
    i++;
```

The body is the empty statement.

**The question to ask every time**

*Which variable in the test does the body change — and does it move towards the limit?*

# `break` and `continue` — recap

```
int i = 0;

while (i < 20) {
    i++;
    if (i % 3 == 0) continue;  // skip
    if (i > 8)      break;     // stop
    cout << i << " ";
}
cout << "| end i = " << i;
```

# `break` and `continue` — recap

```
int i = 0;

while (i < 20) {
    i++;
    if (i % 3 == 0) continue;  // skip
    if (i > 8)      break;     // stop
    cout << i << " ";
}
cout << "| end i = " << i;
```

**Output**

```
1 2 4 5 7 8 | end i = 10
```

`continue` jumps to the next test; `break` leaves the loop.
`i = 9` is skipped by `continue`, so the `break` is only reached at `i = 10`.

# Your turn A

```
int x = 1, c = 0;

while (x < 50) {
    x *= 3;
    c++;
}
cout << x << " " << c;
```

# Your turn A

```
int x = 1, c = 0;

while (x < 50) {
    x *= 3;
    c++;
}
cout << x << " " << c;
```

**Output**

```
81 4
```

x takes 3, 9, 27, 81. The test is checked *before* each pass, so the value that fails it — 81 — is the one the body just produced.

# Your turn B

```
int n = 2436, s = 0;

while (n > 0) {
    s += n % 10;
    n /= 10;
}
cout << s;
```

# Your turn B

```
int n = 2436, s = 0;

while (n > 0) {
    s += n % 10;
    n /= 10;
}
cout << s;
```

**Output**

```
15
```

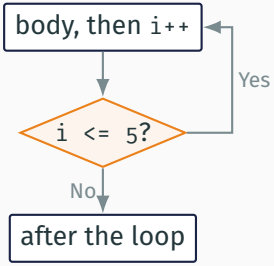
`%` peels off the last digit, `/` throws it away: $6 + 3 + 4 + 2$.
This is the **digit loop** — remember it, it appears in every exam.

# do-while

# do–while: run first, ask afterwards

```
int i = 1;

do {
    cout << i << " ";
    i++;
} while (i <= 5);
```




**Output**

```
1 2 3 4 5
```

## Two details that cost marks

The test is at the bottom, so the body always runs **at least once**. And the closing `while (...)` needs a semicolon — this is the one loop that ends with one.

# The difference shows up at zero

**`while`** — with `n = 0`

```
int i = 1;
while (i <= n) {
    cout << i << " ";
    i++;
}
```

Prints nothing. The test fails immediately.

**`do-while`** — with `n = 0`

```
int i = 1;
do {
    cout << i << " ";
    i++;
} while (i <= n);
```

Prints `1`. The body ran before anyone asked.

**So choose by the question you are answering**

"Maybe zero times" → `while`. "At least once, then decide" → `do-while`.

# Where `do–while` is the right tool

```
int age;

do {
    cout << "Enter age (0-120): ";
    cin >> age;
} while (age < 0 || age > 120);

cout << "Thank you" << endl;
```

**Why it fits**

You must ask *once* before you can possibly know whether the answer is valid.

Written with `while`, the prompt and the read have to be duplicated — once before the loop, once inside.

Read the condition as the *reason to ask again*: “while the answer is still wrong”.

# The classic: a menu that repeats

```
int choice;

do {
    cout << "1 Deposit  2 Balance  0 Exit: ";
    cin >> choice;

    switch (choice) {
        case 1: cout << "Deposit\n";  break;
        case 2: cout << "Balance\n";  break;
        case 0: cout << "Bye\n";      break;
        default: cout << "Unknown\n";
    }
} while (choice != 0);
```

## Where it breaks

If the exit value is never read — a typo in the condition, say `choice != 9` — the menu never ends.

# Your turn A

```
int i = 5;

do {
    cout << i << " ";
    i--;
} while (i > 2);

cout << "| i = " << i;
```

# Your turn A

```
int i = 5;

do {
    cout << i << " ";
    i--;
} while (i > 2);

cout << "| i = " << i;
```

**Output**

```
5 4 3 | i = 2
```

Three passes. After printing 3 the body makes `i` 2, and `2 > 2` is false.

# Your turn B

```
int n = 0, count = 0;

do {
    count++;
    n += count;
} while (n < 10);

cout << count << " " << n;
```

# Your turn B

```
int n = 0, count = 0;

do {
    count++;
    n += count;
} while (n < 10);

cout << count << " " << n;
```

**Output**

```
4 10
```

`n` takes $1, 3, 6, 10$ — the running sum of $1 + 2 + 3 + 4$. At 10 the test `10 < 10` fails.

# for

# `for`: the three parts, on one line

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}
```

| Part | In the header | When it runs |
|---|---|---|
| initialise | `int i = 1` | once, before anything else |
| test | `i <= 5` | before every pass — so zero passes is possible |
| update | `i++` | after every pass, then back to the test |

Nothing new — the same three parts as `while`, collected where you cannot forget one.

# `for` and `while` are the same loop

**`while`**

```
int total = 0;
int i = 1;
while (i <= n) {
    total += i;
    i++;
}
```

**`for`**

```
int total = 0;
for (int i = 1; i <= n; i++) {
    total += i;
}
```

**The rule of thumb**

Use `for` when you know how many passes there will be — counting, stepping, walking over positions. Use `while` when the end depends on the data — a sentinel, a user answer, a condition that may never hold.

The `i` declared in the header exists *only* inside the loop.

# Trace it

```
int p = 1;

for (int i = 1; i <= 4; i++) {
    p *= i;
    cout << i << ":" << p << " ";
}
```

**Output**

```
1:1 2:2 3:6 4:24
```

| Pass | i | i <= 4 | p |
|---|---|---|---|
| 1 | 1 | true | 1 |
| 2 | 2 | true | 2 |
| 3 | 3 | true | 6 |
| 4 | 4 | true | 24 |
| – | 5 | false | 24 |

One row per pass, plus the row where the test fails. That last row is what people forget.

# Four `for` loops worth knowing

```cpp
// count down
for (int i = 5; i >= 1; i--)
    cout << i << " ";        // 5 4 3 2 1

// step by two
for (int i = 0; i <= 10; i += 2)
    cout << i << " ";        // 0 2 4 6 8 10
```

```cpp
// walk the alphabet
for (char c = 'a'; c <= 'e'; c++)
    cout << c;               // abcde

// accumulate
int f = 1;
for (int i = 1; i <= 5; i++)
    f *= i;                  // f = 120
```

The counter need not be an `int`, and need not go up by one — but it must *move towards* the test.

# A complete program: average of n marks

```
int n;
cout << "How many marks? ";
cin >> n;

int total = 0;
for (int i = 1; i <= n; i++) {
    int mark;
    cin >> mark;
    total += mark;
}

cout << fixed << setprecision(2)
     << "Average: " << total / (n * 1.0) << endl;
```

**Input** `4 80 95 70 83`

```
Average:  82.00
```

## Two things to notice

`mark` is declared *inside* the loop — a fresh one each pass, which is exactly right.

`total / n` would be integer division. One `* 1.0` saves the fraction.

# Three `for` traps

### 1. The stray semicolon

```
for (int i = 1; i <= 5; i++);
    cout << i << " ";
```

The loop body is empty — and this will not even compile, because `i` no longer exists on the next line.

### 2. Off by one

```
for (int i = 1; i < 5; i++)   // 4 passes
for (int i = 1; i <= 5; i++)  // 5 passes
```

### 3. Changing the counter inside

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
    i++;              // now it skips
}                     // prints 1 3 5
```

### And never count with a `double`

`for (double x = 0; x != 1.0; x += 0.1)` never stops — 0.1 is not exact in binary.

# Your turn A

```
int total = 0;

for (int i = 1; i <= 6; i++) {
    if (i % 2 == 0) {
        total += i;
    }
}
cout << total;
```

# Your turn A

```
int total = 0;

for (int i = 1; i <= 6; i++) {
    if (i % 2 == 0) {
        total += i;
    }
}
cout << total;
```

**Output**

```
12
```

Only the even values are added: $2 + 4 + 6$. The odd passes still happen — they just do nothing.

# Your turn B

```
for (int i = 10; i > 0; i -= 3) {
    cout << i << " ";
}
cout << "| done";
```

# Your turn B

```
for (int i = 10; i > 0; i -= 3) {
    cout << i << " ";
}
cout << "| done";
```

**Output**

```
10 7 4 1 | done
```

After 1 the counter becomes $-2$, which fails `i > 0`. A counter does not have to land on the limit — only cross it.

# Nested loops

# A loop inside a loop

```
for (int r = 1; r <= 4; r++) {
    for (int c = 1; c <= 4; c++) {
        cout << setw(4) << r * c;
    }
    cout << endl;      // end of the row
}
```

**Output**

```
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
```

## How to read it

The inner loop runs completely for every single pass of the outer one: $4 \times 4 = 16$ passes in total. The `endl` belongs to the outer loop — that is what ends the row.

# Counting the passes

```
int steps = 0;

for (int i = 1; i <= 3; i++)
    for (int j = 1; j <= 5; j++)
        steps++;

cout << steps;      // 15
```

**Outer × inner**

3 outer passes, each running 5 inner passes: $3 \times 5 = 15$.

**Why it matters**

Nested loops multiply. Two loops of 1 000 are 1 000 000 steps — the first time your program will feel *slow*.

# When the inner loop depends on the outer one

```
for (int r = 1; r <= 5; r++) {
    for (int c = 1; c <= r; c++) {
        cout << "*";
    }
    cout << endl;
}
```

**Output**

```
*
**
***
****
*****
```

**The trick**

The inner limit is $r$, not a constant — so row 1 prints one star and row 5 prints five. Every shape exercise you will ever get is this idea plus arithmetic on $r$.

# Which loop should I use?

| Loop | Test is | Use it when |
|---|---|---|
| `while` | before the body | the number of passes depends on the data; zero passes is legal |
| `do-while` | after the body | the body must run at least once — input validation, menus |
| `for` | before the body | you know how many passes: counting, stepping, tables |

**All three are interchangeable**

Any loop can be written with any of them. Choose the one that makes the *reader* see the exit condition fastest.

# Your turn A

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= i; j++) {
        cout << j;
    }
    cout << endl;
}
```

# Your turn A

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= i; j++) {
        cout << j;
    }
    cout << endl;
}
```

**Output**

```
1
12
123
```

The inner loop prints `j`, not `*` — so each row counts up to the row number.

# Your turn B

```
int count = 0;

for (int i = 1; i <= 3; i++)
    for (int j = 1; j <= 4; j++)
        if ((i + j) % 2 == 0)
            count++;

cout << count;
```

# Your turn B

```
int count = 0;

for (int i = 1; i <= 3; i++)
    for (int j = 1; j <= 4; j++)
        if ((i + j) % 2 == 0)
            count++;

cout << count;
```

**Output**

```
6
```

12 pairs in total; `i + j` is even in exactly half of them — two per row.

# Functions

# The problem functions solve

```
// area of three circles
double a1 = 3.14159 * 2 * 2;
double a2 = 3.14159 * 5 * 5;
double a3 = 3.14159 * 7 * 7;
```

The same idea written three times. Change the formula and you must change it in three places — and you will miss one.

## Write it once, name it

```
double area(double r) {
    return 3.14159 * r * r;
}

double a1 = area(2);
double a2 = area(5);
```

This is the decomposition idea from Week 2 — break a problem into smaller pieces — now as real C++.

# Functions you did not write: `#include <cmath>`

```cpp
#include <cmath>

cout << sqrt(81) << endl;      // 9
cout << pow(2, 10) << endl;    // 1024
cout << abs(-7) << endl;       // 7
cout << floor(7.8) << endl;    // 7
cout << ceil(7.2) << endl;     // 8
cout << round(2.6) << endl;    // 3
```

`sqrt`, `pow` and the rest return a `double`, so `pow(2, 10)` is `1024.0` — printed as `1024`.

# The anatomy of your own function

```
int square(int x) {
    return x * x;
}
```

| Piece | Here | What it means |
|---|---|---|
| return type | `int` | the type of the value handed back |
| name | `square` | what you write to call it |
| parameter | `int x` | a variable that receives the caller's value |
| body | `{ ...}` | the work, run only when called |
| `return` | `x * x` | hands the value back and ends the function |

A function that returns nothing has the return type `void`.

# A complete program with a function

```cpp
#include <iostream>
using namespace std;

int square(int x) {        // definition
    return x * x;
}

int main() {
    int n;
    cin >> n;

    cout << square(n) << endl;      // call
    cout << square(3) + square(4);  // call, call
    return 0;
}
```

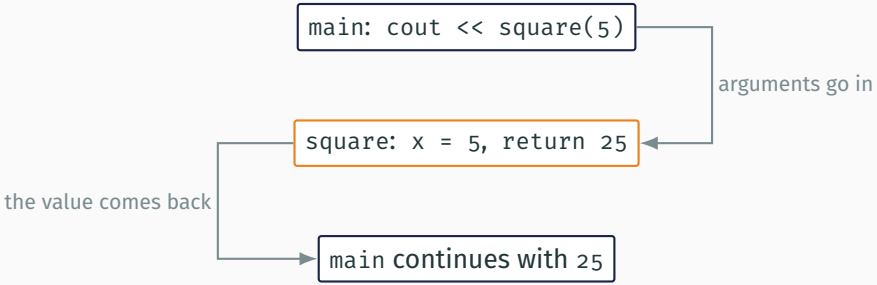
**Input 5**

```
25
25
```

**Order matters — for now**

The definition appears above `main`, because a name must be known before it is used.

# What happens on a call




`main` pauses while `square` runs. The parameter `x` exists only during that call — it disappears when the function returns.

# `return` is not `cout`

**Returns a value — reusable**

```
int twice(int x) {
    return 2 * x;
}

int y = twice(5) + twice(3);  // 16
```

**Prints — and that is all**

```
void twice(int x) {
    cout << 2 * x;
}

int y = twice(5) + twice(3);  // ERROR
```

**The rule**

A function that `returns` a value can be used *inside an expression*. A function that only prints cannot — it gives nothing back to compute with.

Calculate in the function, print in `main`: that way the same function serves every caller.

# Your turn A

```
int twice(int x) {
    return 2 * x;
}

int main() {
    cout << twice(3) + twice(4);
}
```

# Your turn A

```
int twice(int x) {
    return 2 * x;
}

int main() {
    cout << twice(3) + twice(4);
}
```

**Output**

```
14
```

Two separate calls: $6 + 8$. Each call gets its own `x`, finishes, and hands back one number.

# Your turn B

```
double average(int a, int b) {
    return (a + b) / 2.0;
}

int main() {
    cout << average(7, 8) << " "
         << average(4, 4);
}
```

# Your turn B

```
double average(int a, int b) {
    return (a + b) / 2.0;
}

int main() {
    cout << average(7, 8) << " "
         << average(4, 4);
}
```

**Output**

```
7.5 4
```

`/ 2.0` keeps the fraction. The second call gives `4.0`, and `cout` prints it as `4`.

# Wrap-up

# What to take away

## Loops

- Initialise, test, update — all three, every time.
- `while` may run zero times; `do–while` always runs once and ends with a semicolon.
- `for` is for a known number of passes.
- A nested loop multiplies: outer $\times$ inner passes.

## Functions

- A function is a named piece of work with a return type, parameters and a body.
- Calling one pauses `main`, runs the body, and comes back with a value.
- `return` gives a value to the caller; `cout` only shows it on screen.

**Questions?**